



Politecnico
di Bari

DIPARTIMENTO
INTERATENEO
DI FISICA



Dottorato in Fisica – XXXIX ciclo - 2024

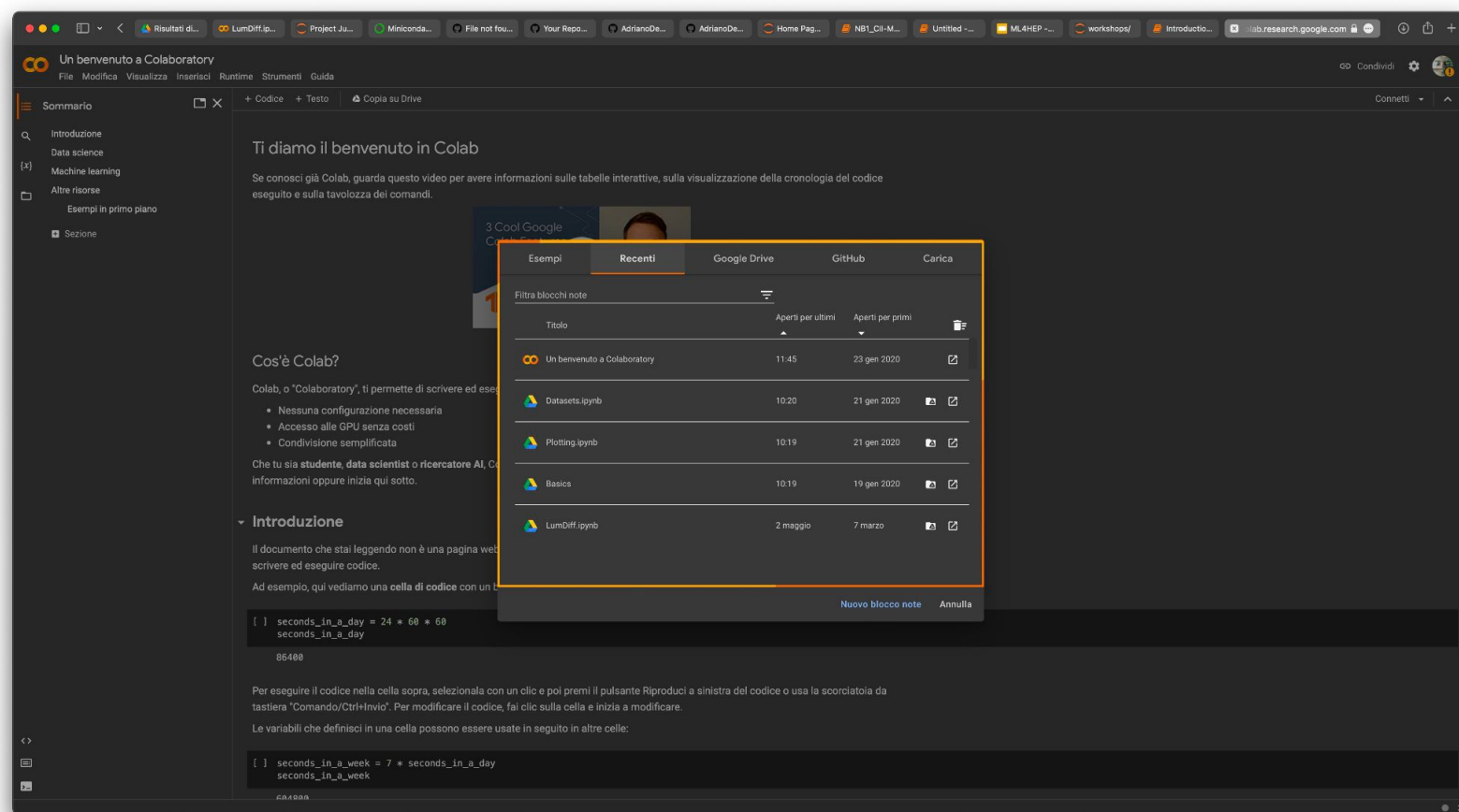
Machine Learning techniques for particle physics

Federica Maria Simone (Poliba & INFN) - federica.simone@poliba.it

Adriano Di Florio (CC-IN2P3) - adriano.di.florio@cern.ch

Hands-on!

GitHub repo: https://github.com/fsimone91/course_ml4hep/tree/main
Go to: <http://colab.research.google.com/>
Import notebook from github



Pre-exercises

If you never played with python, you need to go through the following notebooks:

https://github.com/fsimone91/course_ml4hep/tree/main/notebooks/intro_notebooks

Exercise on gradient descent visualization

https://github.com/fsimone91/course_ml4hep/blob/main/notebooks/lesson2/4_Gradient_Descend_Visualize.ipynb

Jupyter notebook: install locally

1) Install [MiniConda](#)

Miniconda

Miniconda is a free minimal installer for conda. It is a small, bootstrap version of Anaconda that includes only conda, Python, the packages they depend on, and a small number of other useful packages, including pip, zlib and a few others. Use the `conda install` command to install 720+ additional conda packages from the Anaconda repository.

[See if Miniconda is right for you.](#)

System requirements

- License: Free use and redistribution under the terms of the [EULA for Miniconda](#).
- Operating system: Windows 8 or newer, 64-bit macOS 10.13+, or Linux, including Ubuntu, RedHat, CentOS 7+, and others.
- If your operating system is older than what is currently supported, you can find older versions of the Miniconda installers in our [archive](#) that might work for you.
- System architecture: Windows- 64-bit x86, 32-bit x86; macOS- 64-bit x86 & Apple M1 (ARM64); Linux- 64-bit x86, 64-bit aarch64 (AWS Graviton2), 64-bit IBM Power8/Power9, s390x (Linux on IBM Z & LinuxONE).
- The `linux-aarch64` Miniconda installer requires `glibc >= 2.26` and thus will not work with CentOS 7, Ubuntu 16.04, or Debian 9 ("stretch").
- Minimum 400 MB disk space to download and install.

On Windows, macOS, and Linux, it is best to install Miniconda for the local user, which does not require administrator permissions and is the most robust type of installation. However, if you need to, you can install Miniconda system wide, which does require administrator permissions.

Latest Miniconda Installer Links

Latest - Conda 23.3.1 Python 3.10.10 released April 24, 2023

Platform	Name	SHA256 hash
Windows	Miniconda3 Windows 64-bit	307194e1f12bbeb52b083634e09cc67db4f7980bd542254b43d3309ea77c358
	Miniconda3 Windows 32-bit	4fb64e6c9c28b08beab16994bfb4829118ea3145baa06bda5344174ab65d462
macOS	Miniconda3 macOS Intel x86 64-bit bash	5ebc78b664b7da9e14ade330534cc98283b38c6b10ad9cf8b9cc4153f8104
	Miniconda3 macOS Intel x86 64-bit pkg	cca31a0f1e5394f2b739726dc2251c2a19afdf689c13a25668887ba706cbe58
	Miniconda3 macOS Apple M1 64-bit bash	9d1d12573339c49050b0d5a848af9ff6c32d33c3de1b3db478c01870eb083d64
	Miniconda3 macOS Apple M1 64-bit pkg	6997472c5ff90a772eb77e6397f4e3e227736c03a7f7b039da3306cc7facb75d
Linux	Miniconda3 Linux 64-bit	ae279dbaea7f67940f18aad17ebe5f6aac97487c7c83466ff081f4819e5a651
	Miniconda3 Linux-aarch64 64-bit	6950c7b1f4f65ce0b87ee1a2e684837771ae7b2e6044e0da9e915d1d0ee6c924c
	Miniconda3 Linux-ppc64le 64-bit	b3de538cd542bc4f5a2f2d2a793862806e04f0e1459755f3cefe64763e51d16
	Miniconda3 Linux-s390x 64-bit	ed4f51afc967e921ff5721151f567a4c43c4288ac93ec2303c62380bc4891de8

Windows installers

Windows

Jupyter notebook: install locally

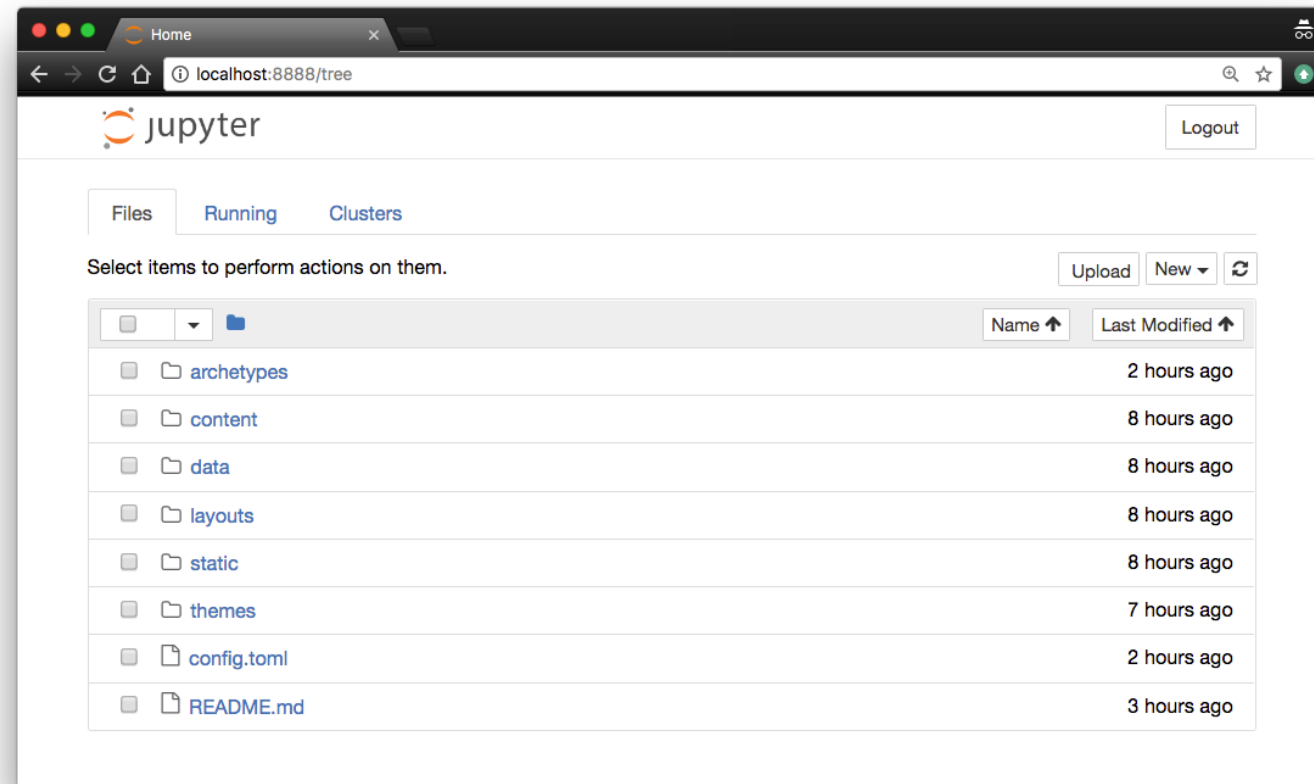
2) Create a python environment

```
conda create -n myenv anaconda python=3
```

3) Install jupyter-notebook

```
conda install jupyter
```

3) Run it! (Typically on <http://localhost:8888>)



Jupyter notebook using RECAS

Instructions:

https://jvino.github.io/cluster-hpc-gpu-guides/interactive_services/jupyter-lab-for-recas-users/



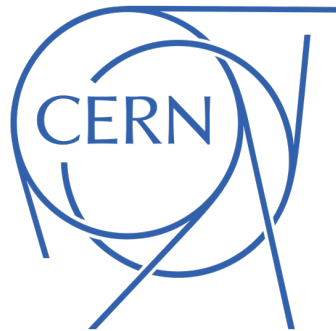
RECAS BARI



Jupyter notebook using CERN SWAN

If you have a CERN account you also have access to SWAN (Service for Web based ANalysis). SWAN is a platform to perform interactive data analysis in the cloud.

<https://swan.web.cern.ch/swan/>



Dense Neural Networks: hands-on!

simpli|learn



VS



VS



KERAS

TENSORFLOW

PYTORCH



[Keras](#) is an effective high-level neural network **Application Programming Interface (API)** written in Python.

This open-source neural network library is designed to provide fast experimentation with deep neural networks, and it can **run on top of CNTK, TensorFlow, and Theano**.

Keras focuses on being modular, user-friendly, and extensible. It doesn't handle low-level computations; instead, it hands them off to another library called the Backend.



[TensorFlow](#) is an end-to-end open-source deep learning framework developed by Google and released in 2015. It is known for documentation and training support, scalable production and deployment options, multiple abstraction levels, and support for different platforms, such as Android.

TensorFlow is a **symbolic math library** used for neural networks and is best suited for dataflow programming across a range of tasks. It offers multiple abstraction levels for building and training models.

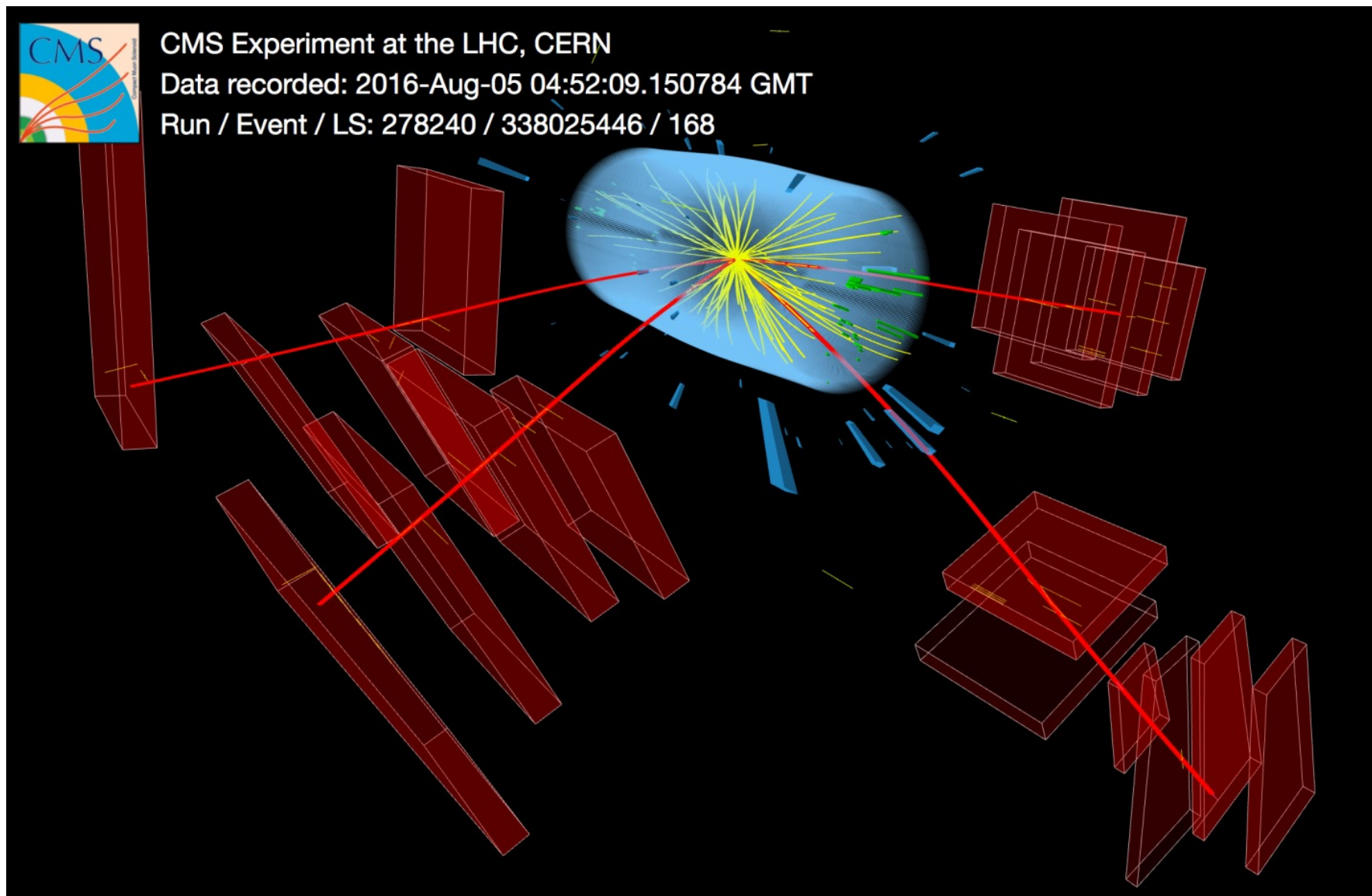
Also, TensorFlow has adopted Keras API.



[PyTorch](#) is a relatively new deep learning framework based on Torch. Originally developed by Meta AI and now part of the Linux Foundation umbrella, it is an optimized tensor library for deep learning using GPUs and CPUs.

PyTorch has a reputation for simplicity, ease of use, flexibility, efficient memory usage, and dynamic computational graphs. It also feels native, making coding more manageable and increasing processing speed.

First exercise: $H \rightarrow ZZ \rightarrow 4l$ analysis

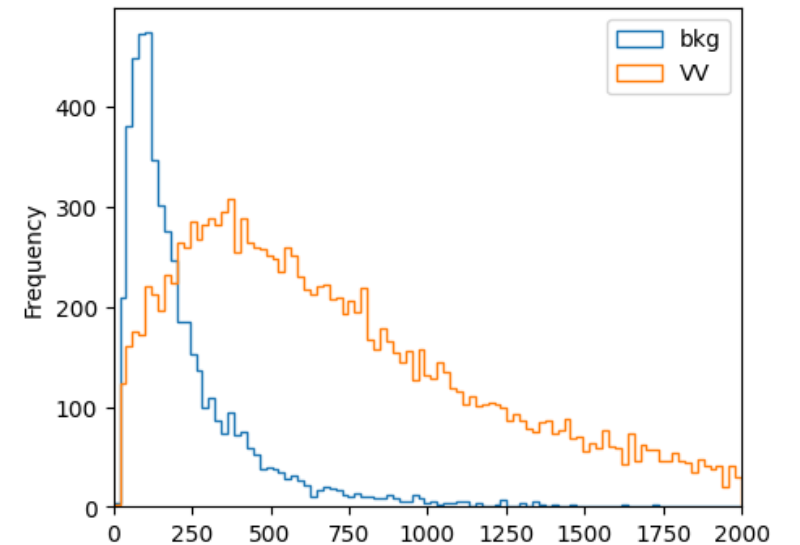
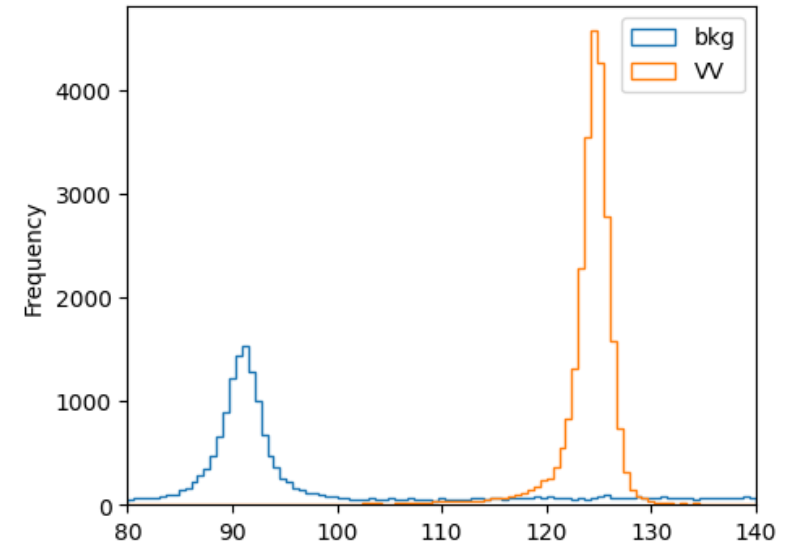
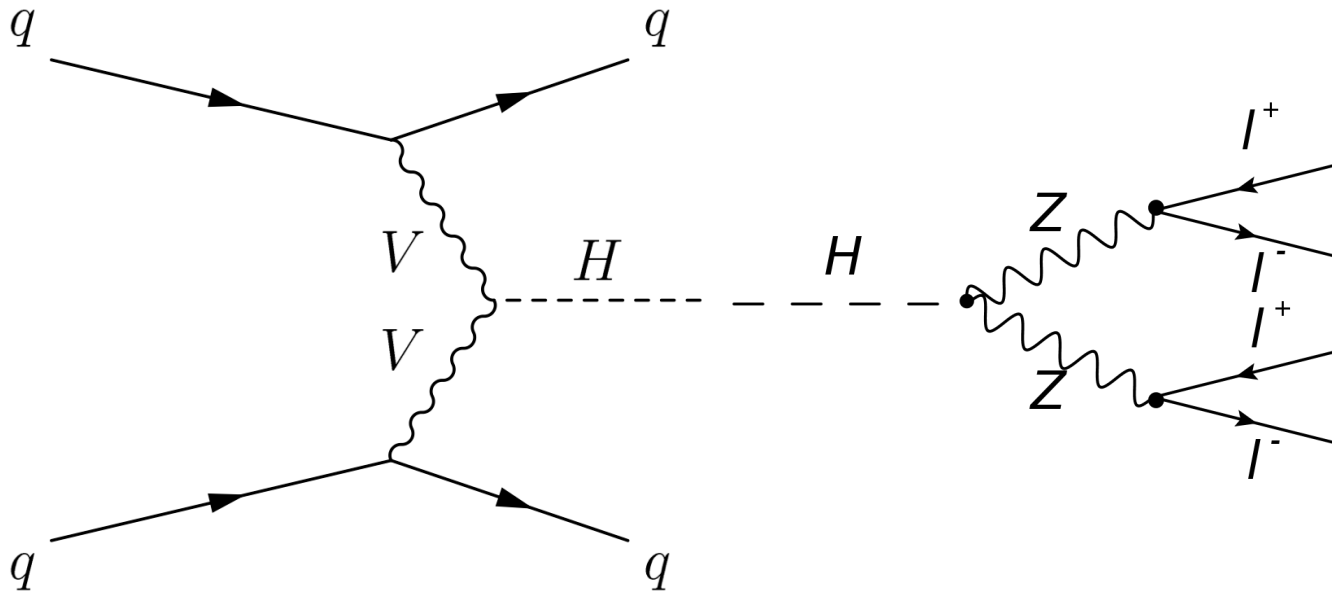


First exercise: $H \rightarrow ZZ \rightarrow 4l$ analysis

Higgs production mechanism: VBF

Higgs decay: $H \rightarrow ZZ \rightarrow 4l$

Goal: discriminate VBF Higgs and standard model background 4 muon events.



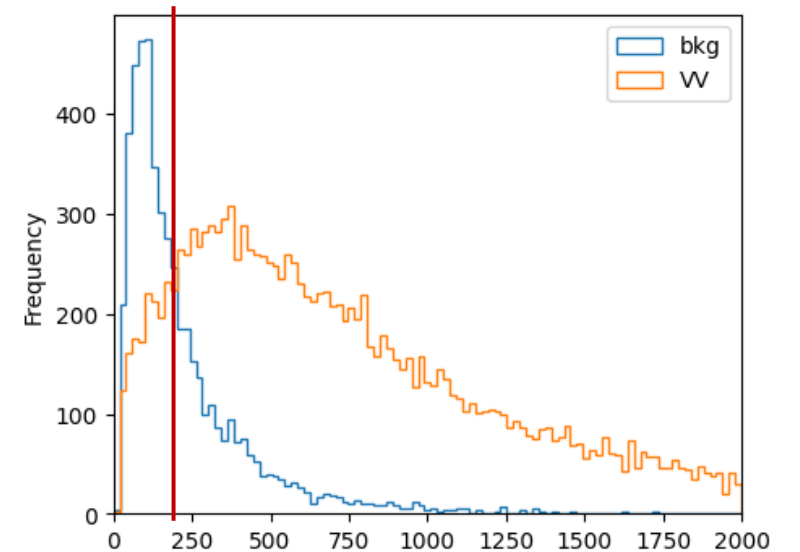
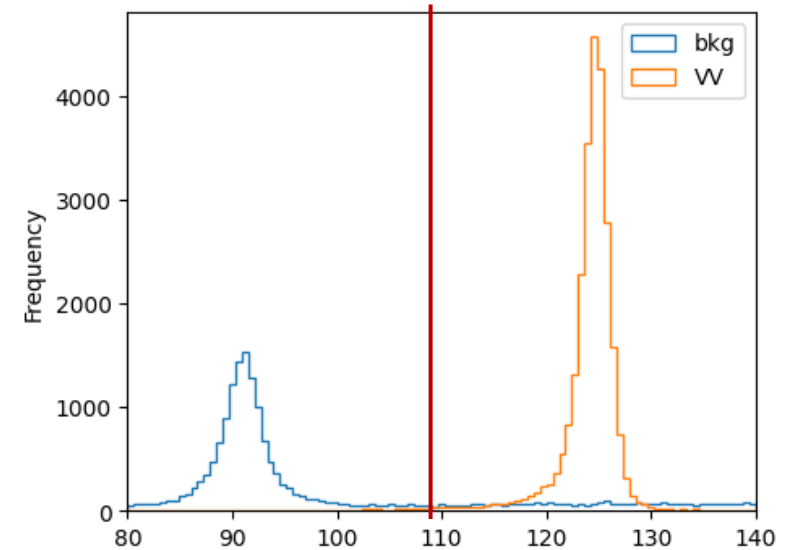
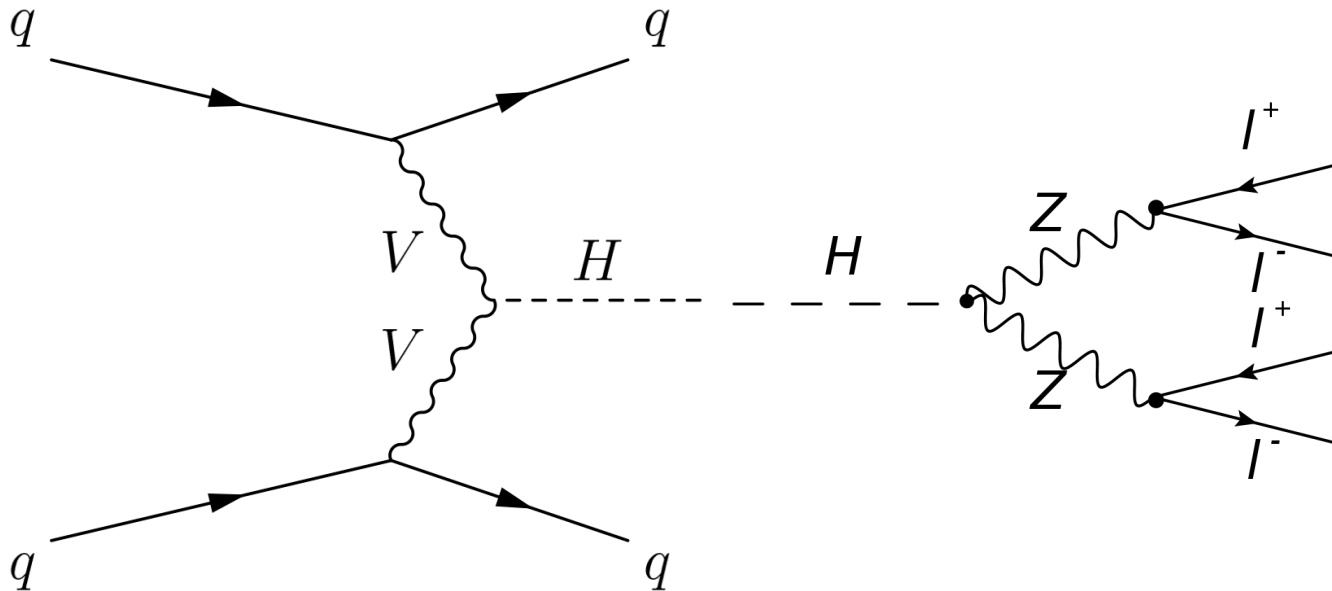
First exercise: $H \rightarrow ZZ \rightarrow 4l$ analysis

Higgs production mechanism: VBF

Higgs decay: $H \rightarrow ZZ \rightarrow 4l$

Goal: discriminate VBF Higgs and standard model background 4 muon events.

“Cut-based” analysis: classify signal and background events based on thresholds on some observables

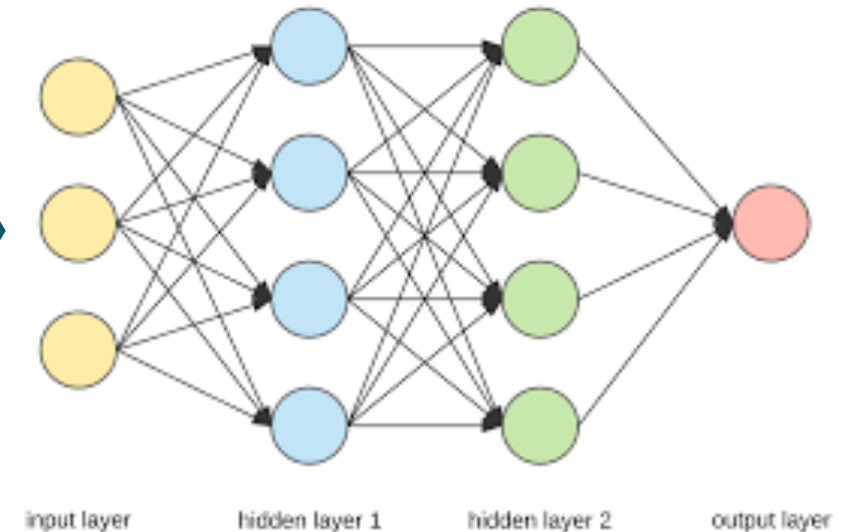
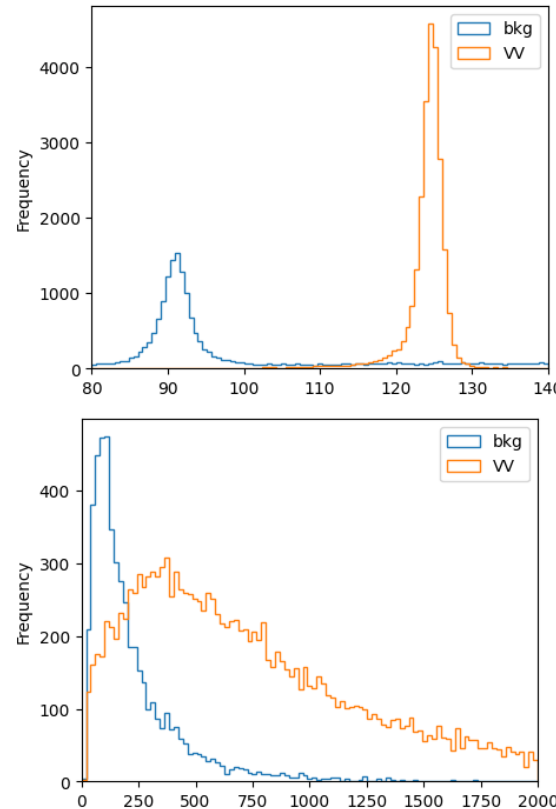


First exercise: $H \rightarrow ZZ \rightarrow 4l$ analysis

Let's solve this with ML!

Notebook 1:

- Download the rootfiles
- Convert Ttree into numpy dataframe
- Plot input features using matplotlib



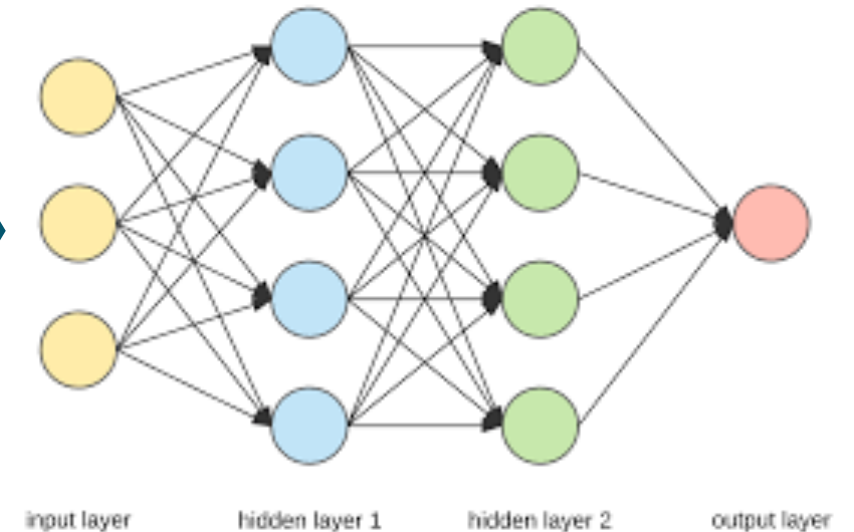
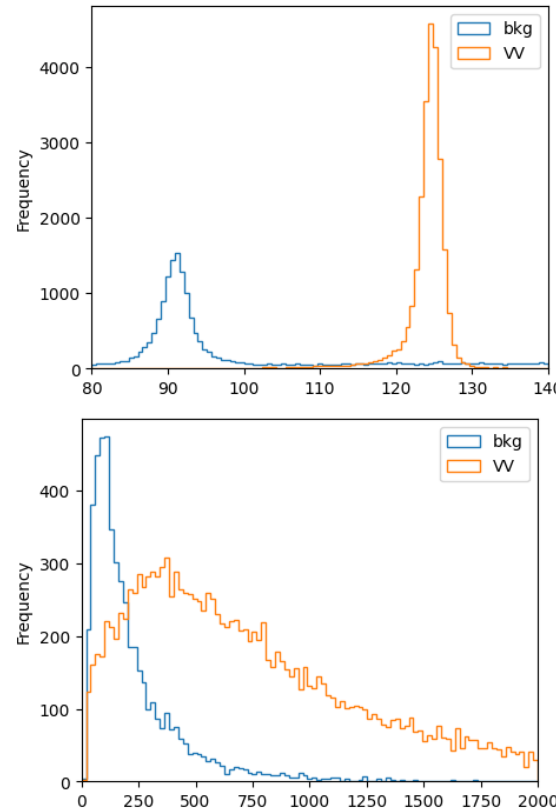
https://github.com/fsimone91/course_ml4hep/tree/2024/notebooks/2024/1-datasets-uproot.ipynb

First exercise: $H \rightarrow ZZ \rightarrow 4l$ analysis

Let's solve this with ML!

Notebook 2:

- Define a dense NN in keras+tensorflow
- Train it
- Look at performance using test set
- Play with the hyperparameters



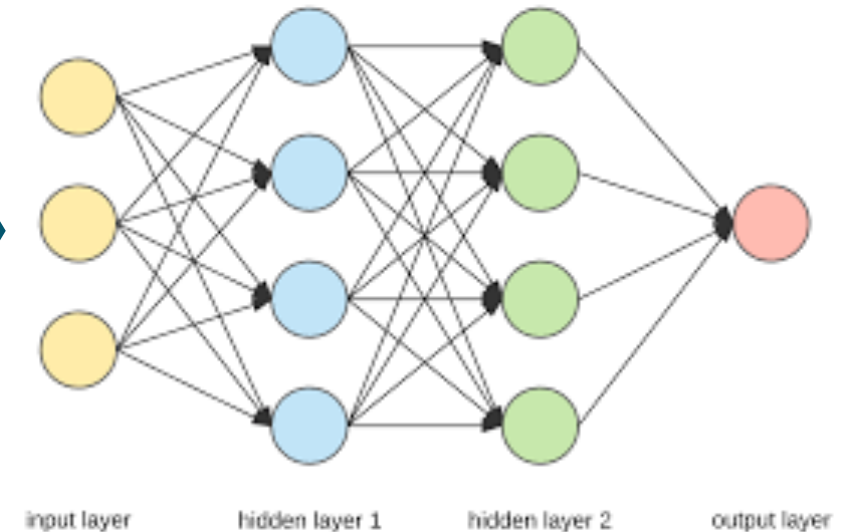
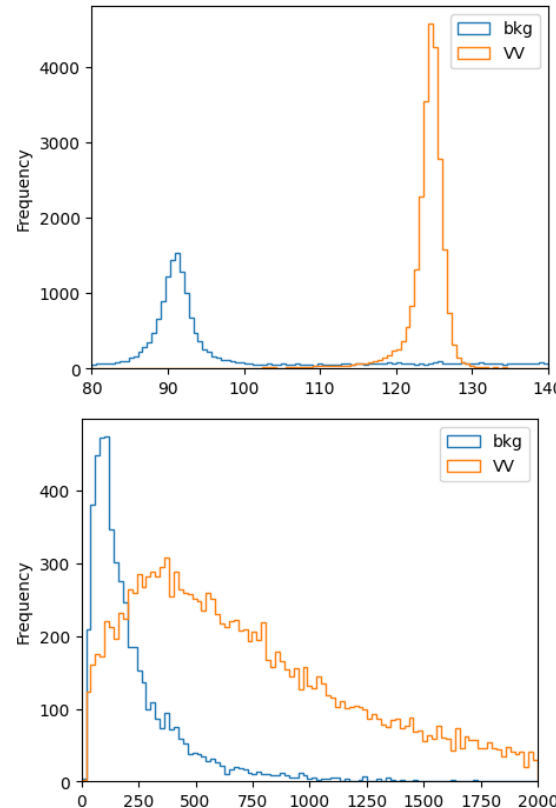
https://github.com/fsimone91/course_ml4hep/tree/2024/notebooks/2024/2.1-dense-keras.ipynb

First exercise: $H \rightarrow ZZ \rightarrow 4l$ analysis

Let's solve this with ML!

Notebook 3:

- Define a dense NN in pytorch
- Train it
- Look at performance using test set
- Play with the hyperparameters



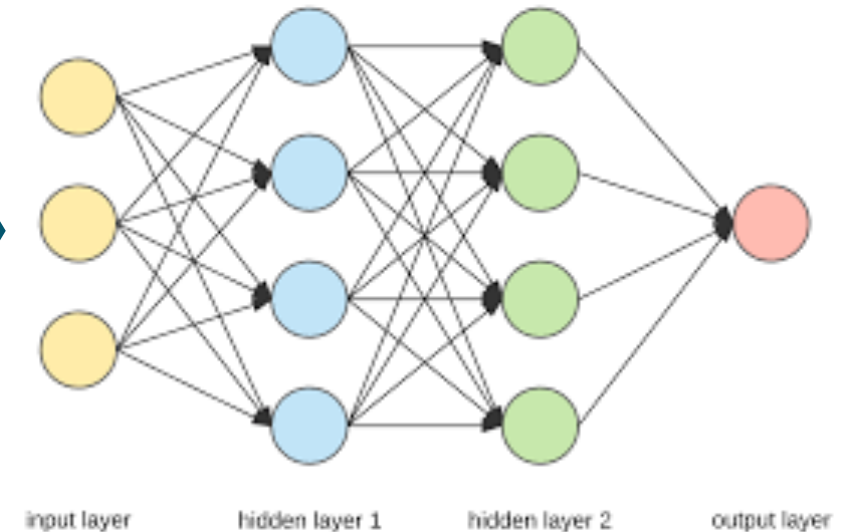
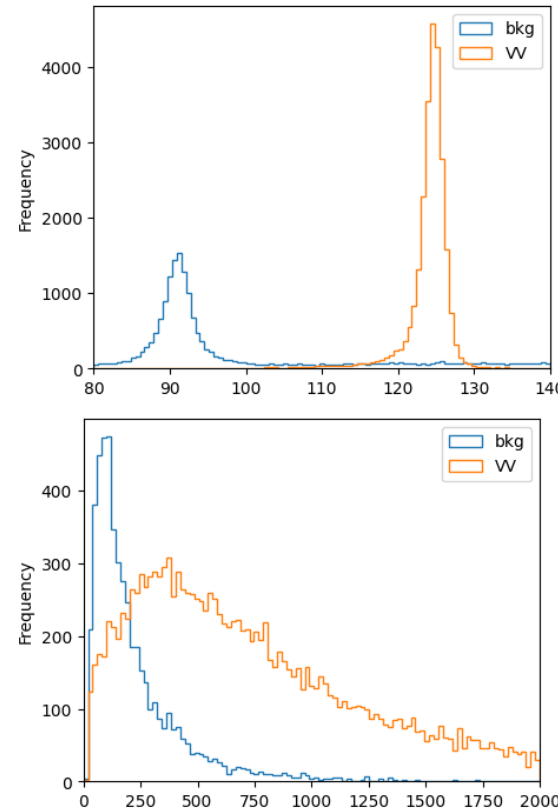
https://github.com/fsimone91/course_ml4hep/tree/2024/notebooks/2024/2.2-dense-pytorch.ipynb

First exercise: $H \rightarrow ZZ \rightarrow 4l$ analysis

Let's solve this with ML!

Extra: notebook 4

- Define a dense NN in keras+tensorflow
- Optimise some hyperparameters using [Scikit-Optimize](#)



https://github.com/fsimone91/course_ml4hep/tree/2024/notebooks/2024/2.3-dense-bayesian-optimization.ipynb

Bayesian optimization with

Gilles Louppe, Manoj Kumar July 2016. Reformatted by Holger Nahrstaedt 2020

Problem statement

We are interested in solving

$$x^* = \underset{\min_x}{\operatorname{arg}} f(x)$$

under the constraints that

- f is a black box for which no closed form is known (nor its gradients);
- f is expensive to evaluate;
- and evaluations of $y = f(x)$ may be noisy.

Disclaimer. If you do not have these constraints, then there is certainly a better optimization algorithm than Bayesian optimization.

Bayesian optimization loop

For $t = 1 : T$:

1. Given observations $(x_i, y_i = f(x_i))$ for $i = 1 : t$, build a probabilistic model for the objective f . Integrate out all possible true functions, using Gaussian process regression.
2. optimize a cheap acquisition/utility function u based on the posterior distribution for sampling the next point. $x_{t+1} = \operatorname{argmin}_x u(x)$ Exploit uncertainty to balance exploration against exploitation.
3. Sample the next observation y_{t+1} at x_{t+1} .

Acquisition functions

Acquisition functions $u(x)$ specify which sample x : should be tried next:

- Expected improvement (default): $-EI(x) = -\mathbb{E}[f(x) - f(x_t^+)]$
- Lower confidence bound: $LCB(x) = \mu_{GP}(x) + \kappa \sigma_{GP}(x)$
- Probability of improvement: $-PI(x) = -P(f(x) \geq f(x_t^+) + \kappa)$

where x_t^+ is the best point observed so far.

In most cases, acquisition functions provide knobs (e.g., κ) for controlling the exploration-exploitation trade-off. - Search in regions where $\mu_{GP}(x)$ is high (exploitation) - Probe regions where uncertainty $\sigma_{GP}(x)$ is high (exploration)